

Reinforcement Learning Policy Evaluation in a Grid World

Wenkang Ji

October 5, 2025

1 Problem Setup

We evaluate two policies within a discrete 5×3 grid world environment. The setup is defined as follows:

- **Environment:** A grid of size 5×3 . The start state is $(0, 2)$, the goal state is $(2, 2)$, and there are forbidden states at $\{(2, 1), (1, 2), (3, 1)\}$.
- **Action Space:** The agent can choose from five actions: $(0, 1)$ (down), $(1, 0)$ (right), $(0, -1)$ (up), $(-1, 0)$ (left), and $(0, 0)$ (stay).
- **Reward Function:** The agent receives a reward of $R_{\text{target}} = +1$ for reaching the goal state, $R_{\text{forbidden}} = -1$ for attempting to enter a forbidden state, and $R_{\text{step}} = 0$ for all other valid moves.
- **Discount Rate:** Future rewards are discounted by a factor of $\gamma = 0.9$.

2 Policy 1: Deterministic Policy

The first policy is deterministic and optimal, generated by a reverse Breadth-First Search (BFS) from the goal state. For any given state, this policy selects the action that moves the agent along a shortest path to the target.

2.1 Policy Visualization

The policy is visualized in Figure 1, where arrows indicate the deterministic action for each state.

2.2 Policy Dynamics: r^π and P^π

The policy's expected one-step rewards, r^π , and state transition probabilities, P^π , are shown below.

Reward Vector r^π :

State	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11
Reward	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0

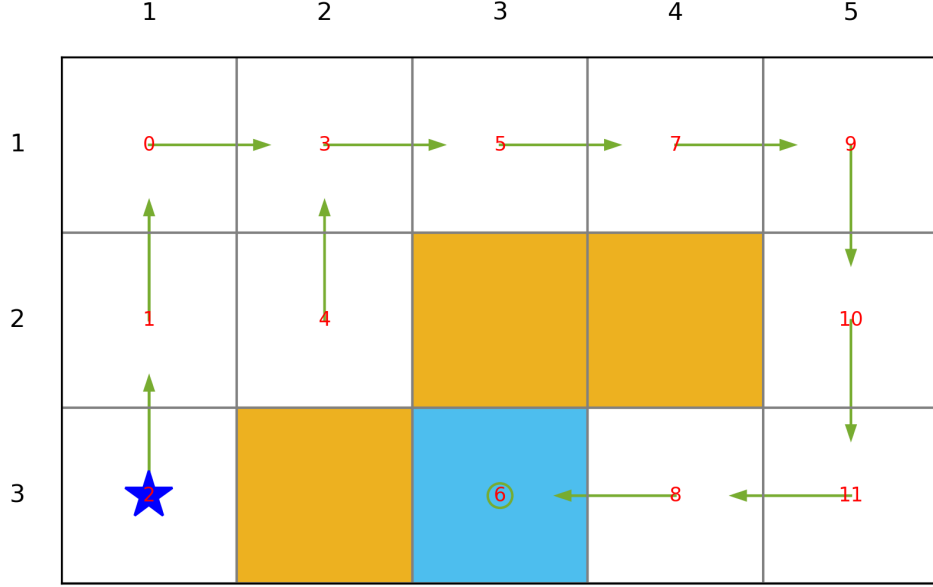


Figure 1: Deterministic policy visualized with arrows on the grid.

Transition Matrix P^π :

From/To	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11
S0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
S1	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
S2	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
S3	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
S4	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
S5	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
S6	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0
S7	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
S8	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0
S9	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
S10	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0
S11	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0

2.3 Closed-Form Evaluation

The state-value function V^π for a fixed policy can be found by directly solving the Bellman expectation equation, which forms a system of linear equations:

$$V^\pi = r^\pi + \gamma P^\pi V^\pi \Rightarrow (I - \gamma P^\pi) V^\pi = r^\pi$$

We solve this system for V^π using a numerical linear algebra solver. The resulting values are shown below and plotted in Figure 2.

State Values V^π (Closed-Form):

State	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11
Value	4.783	4.305	3.874	5.314	4.783	5.905	10.0	6.561	10.0	7.29	8.1	9.0

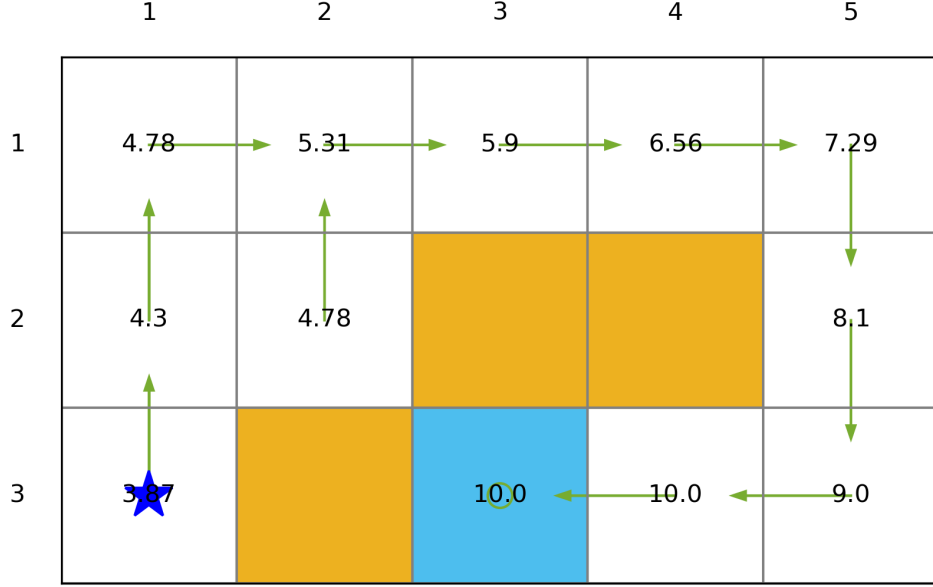


Figure 2: State values for the deterministic policy calculated via the closed-form method.

2.4 Iterative Evaluation

An alternative to the closed-form solution is iterative policy evaluation, which applies the Bellman update rule repeatedly until the value function converges:

$$V^{(k+1)} = r^\pi + \gamma P^\pi V^{(k)}$$

This process is terminated when the maximum change between successive value functions, $\|V^{(k+1)} - V^{(k)}\|_\infty$, falls below a small tolerance. This method converged in 133 iterations. The final values, which match the closed-form solution, are shown below and plotted in Figure 3.

State Values V^π (Iterative, 133 iterations):

State	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11
Value	4.783	4.305	3.874	5.314	4.783	5.905	10.0	6.561	10.0	7.29	8.1	9.0

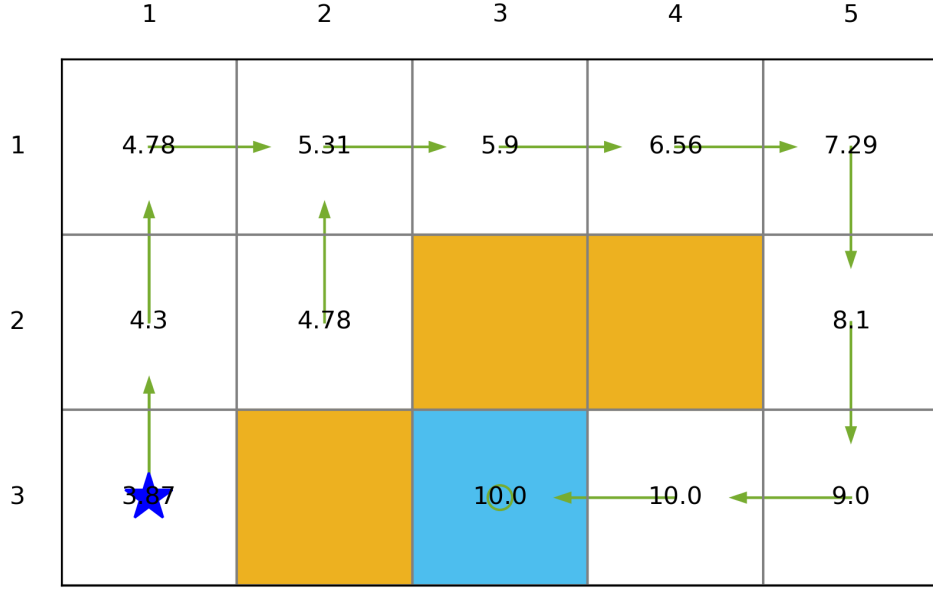


Figure 3: State values for the deterministic policy calculated via the iterative method.

3 Policy 2: Stochastic Policy

The second policy is stationary and stochastic. In non-terminal states, it selects actions based on a fixed probability distribution with a bias towards moving right (0.7), down/up (0.1 each), left (0.05), and stay (0.05).

3.1 Policy Visualization

Figure 4 illustrates the stochastic policy. The size of the arrows is proportional to the probability of taking that action in a given state.

3.2 Policy Dynamics: r^π and P^π

The expected one-step rewards and transition probabilities for the stochastic policy are as follows.

Reward Vector r^π :

State	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11
Reward	-0.25	-0.15	-0.85	-0.2	-0.8	-0.3	1.0	-0.3	-0.25	-0.8	-0.75	-0.8

Transition Matrix P^π :

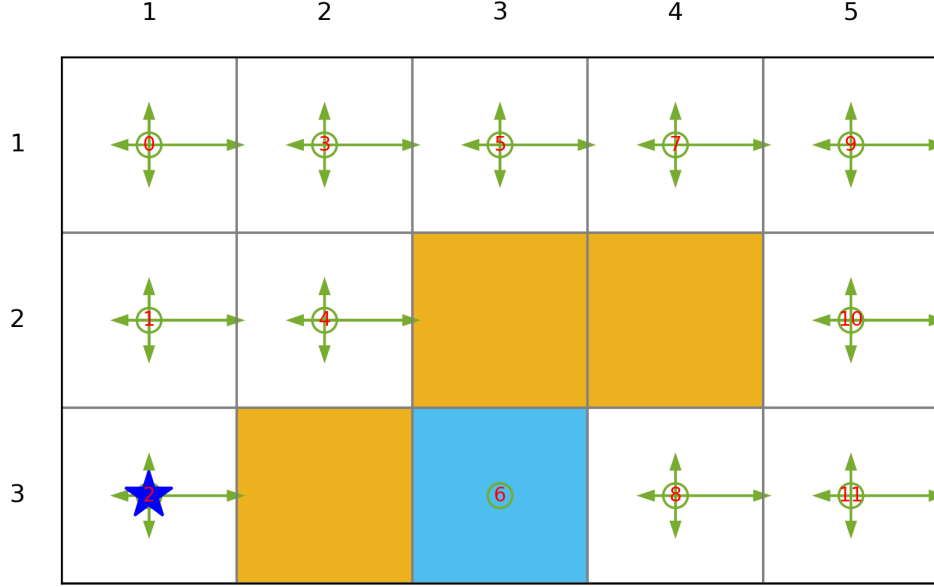


Figure 4: Stochastic policy visualized on the grid. Arrow size corresponds to action probability.

From/To	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11
S0	0.2	0.1	0.0	0.7	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
S1	0.1	0.1	0.1	0.0	0.7	0.0	0.0	0.0	0.0	0.0	0.0	0.0
S2	0.0	0.1	0.9	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
S3	0.05	0.0	0.0	0.15	0.1	0.7	0.0	0.0	0.0	0.0	0.0	0.0
S4	0.0	0.05	0.0	0.1	0.85	0.0	0.0	0.0	0.0	0.0	0.0	0.0
S5	0.0	0.0	0.0	0.05	0.0	0.25	0.0	0.7	0.0	0.0	0.0	0.0
S6	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0
S7	0.0	0.0	0.0	0.0	0.0	0.05	0.0	0.25	0.0	0.7	0.0	0.0
S8	0.0	0.0	0.0	0.0	0.0	0.0	0.05	0.0	0.25	0.0	0.0	0.7
S9	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.05	0.0	0.85	0.1	0.0
S10	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.8	0.1
S11	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.05	0.0	0.1	0.85

3.3 Closed-Form Evaluation

Using the same direct solution method as before, we calculate the state values for the stochastic policy. The results are shown below and plotted in Figure 5.

State Values V^π (Closed-Form):

State	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11
Value	-5.522	-6.174	-7.248	-5.909	-6.626	-6.546	10.0	-6.752	-5.571	-7.555	-7.463	-7.463

3.4 Iterative Evaluation

The iterative algorithm also converged in 133 iterations for the stochastic policy, yielding results consistent with the closed-form method. The values are presented below and in Figure 6.

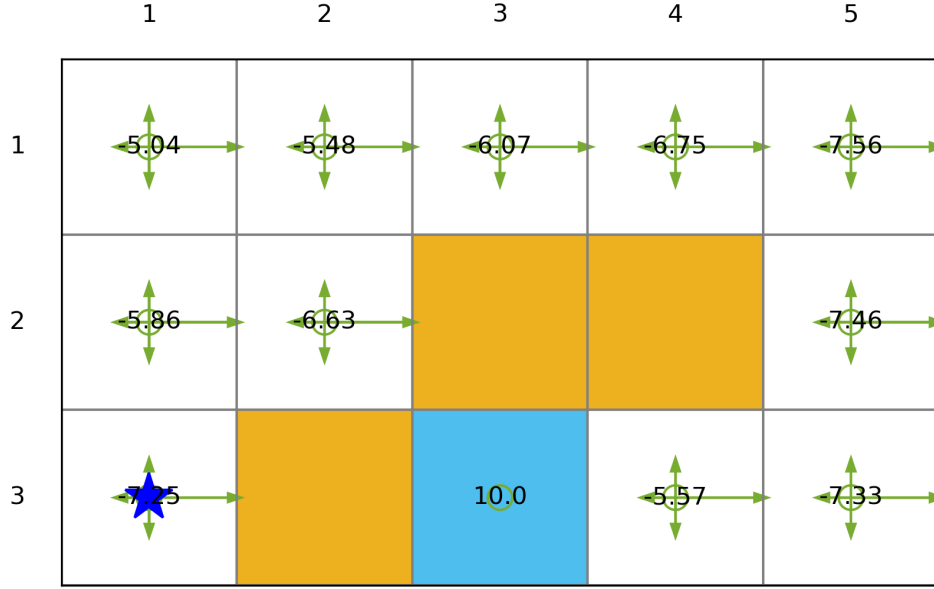


Figure 5: State values for the stochastic policy calculated via the closed-form method.

State Values V^π (Iterative, 133 iterations):

State	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S
Value	-5.522	-6.174	-7.248	-5.909	-6.626	-6.546	10.0	-6.752	-5.571	-7.555	-7.463	-7

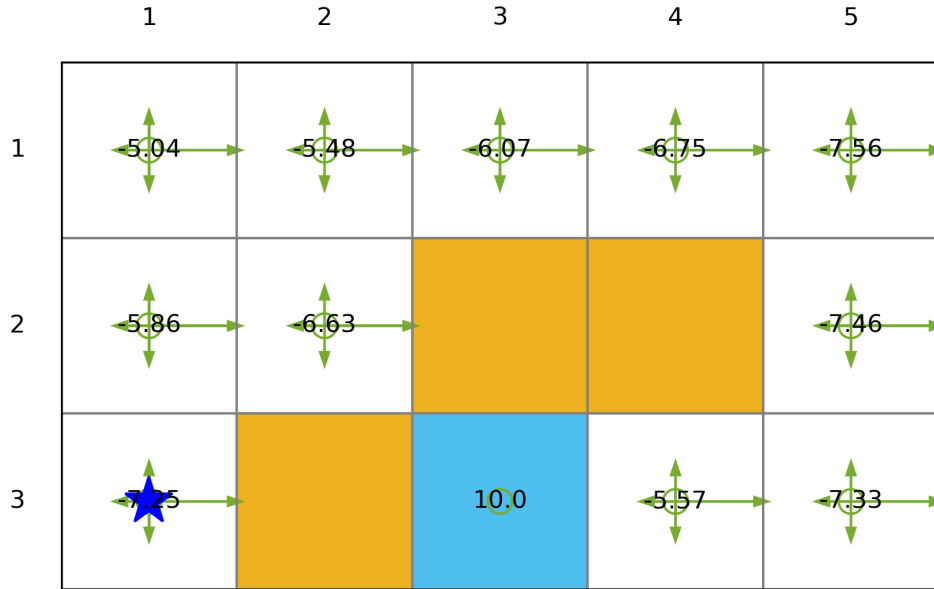


Figure 6: State values for the stochastic policy calculated via the iterative method.

4 Discussion

Comparison of Policies: The results clearly show the superiority of the deterministic, optimal policy. Its state values are all positive, increasing along the shortest paths toward the goal state, which has the maximum value of 10. This reflects the policy's effectiveness in reaching the target and accumulating rewards. In contrast, the stochastic policy performs poorly. Most of its state values are negative because its fixed action probabilities frequently lead the agent into forbidden states, incurring penalties. The negative values propagate throughout the state space, indicating that following this policy is generally undesirable.

Comparison of Algorithms: For both policies, the closed-form and iterative evaluation algorithms produced identical state values, as expected. The closed-form method provides an exact solution by solving a system of linear equations, which is feasible for this small state space. The iterative method converges to the same solution and is often more practical for larger state spaces where directly solving or inverting a large matrix would be computationally prohibitive. The number of iterations (133) shows a reasonably fast convergence for this problem.

5 Key Code Implementation

The core logic for policy evaluation is contained in the `PolicyEvaluator` class, which encapsulates the functionality for computing policy dynamics and evaluating state values using both closed-form and iterative methods.

5.1 Deriving Policy Dynamics

This function computes the reward vector r^π and transition matrix P^π from the environment dynamics and a given policy matrix.

```
1 def _derive_policy_markov_dynamics(self, world: GridWorld) -> Tuple[np.
   ndarray, np.ndarray]:
2     """
3     Compute r_pi and P_pi for a fixed policy:
4         r_pi[s]      = E[R | s, a ~ pi( |s)]
5         P_pi[s,s']   = Pr(s' | s, a ~ pi( |s)]
6     """
7     nS = world.num_states
8     r_pi = np.zeros(nS, dtype=float)
9     P_pi = np.zeros((nS, nS), dtype=float)
10
11     acts = world.action_space
12     s2i = world.state_to_idx
13     i2s = world.idx_to_state
14
15     # Accumulate expectation under action distribution
16     for s in range(nS):
17         xy = i2s[s]
18         pmf = self.policy_matrix[s]
19         if not np.any(pmf):
20             continue
21         for a_idx, prob in enumerate(pmf):
22             if prob <= 0.0:
23                 continue
```

```

24         nxt_xy, rew = world._get_next_state_and_reward(xy, acts[
a_idx])
25         s_next = s2i[nxt_xy]
26         r_pi[s] += prob * rew
27         P_pi[s, s_next] += prob
28
29     return r_pi, P_pi

```

Listing 1: Method to compute r^π and P^π .

5.2 Closed-Form Solution

This function implements the direct, closed-form solution to the Bellman expectation equation.

```

1 def solve_analytically(self, gamma: float) -> np.ndarray:
2     """
3     Solve (I - gamma P)V = r via linear solve.
4     """
5     I = np.eye(self.transition_matrix.shape[0], dtype=float)
6     return np.linalg.solve(I - gamma * self.transition_matrix, self.
reward_vector)

```

Listing 2: Function for closed-form policy evaluation.

5.3 Iterative Solution

This function implements the iterative policy evaluation algorithm, which repeatedly applies the Bellman update until convergence.

```

1 def solve_iteratively(
2     self,
3     gamma: float,
4     tol: float = 1e-6,
5     max_iter: int = 10_000,
6 ) -> Tuple[np.ndarray, int]:
7     """
8     Fixed-point iteration:  $V_{k+1} = r + \gamma P V_k$ 
9     """
10    v = np.zeros_like(self.reward_vector)
11    for k in range(1, max_iter + 1):
12        v_next = self.reward_vector + gamma * self.transition_matrix.
dot(v)
13        if np.max(np.abs(v_next - v)) < tol:
14            return v_next, k
15        v = v_next
16    return v, max_iter

```

Listing 3: Function for iterative policy evaluation.

5.4 Policy Creation

The deterministic and stochastic policies are created using dedicated functions that generate policy matrices.


```

1 def create_optimal_path_policy(world: GridWorld) -> np.ndarray:
2     """
3     Build a deterministic policy by back-propagating shortest steps
4     from the goal using a reverse BFS over the grid.
5     """
6     nS = world.num_states
7     acts = world.action_space
8     pi = np.zeros((nS, len(acts)), dtype=float)
9
10    stay_idx = acts.index((0, 0))
11    goal = tuple(world.target_state)
12
13    # Reverse BFS: record a "next-hop" for each state toward the goal
14    q = deque([goal])
15    next_hop = {goal: None}
16
17    H, W = world.env_size # (rows, cols)
18
19    while q:
20        cur = q.popleft()
21        # consider predecessors that could move into 'cur'
22        for dx, dy in acts:
23            if dx == 0 and dy == 0:
24                continue
25            prev = (cur[0] - dx, cur[1] - dy)
26            if 0 <= prev[0] < H and 0 <= prev[1] < W:
27                if prev not in world.forbidden_states and prev not in
next_hop:
28                    next_hop[prev] = cur
29                    q.append(prev)
30
31    # Turn the shortest-next-hop map into a deterministic policy
32    for s_idx in range(nS):
33        s_xy = world.idx_to_state[s_idx]
34        if s_xy == goal or s_xy in world.forbidden_states or s_xy not
in next_hop:
35            pi[s_idx, stay_idx] = 1.0
36            continue
37
38        nxt = next_hop[s_xy]
39        move = (nxt[0] - s_xy[0], nxt[1] - s_xy[1])
40        if move in acts:
41            a_idx = acts.index(move)
42            pi[s_idx, a_idx] = 1.0
43        else:
44            pi[s_idx, stay_idx] = 1.0
45
46    return pi

```

Listing 4: Deterministic policy creation using reverse BFS.

```

1 def create_weighted_random_policy(world: GridWorld) -> np.ndarray:
2     """
3     Build a stationary stochastic policy with fixed action
probabilities
4     (except terminal/forbidden states that enforce 'stay').
5     """
6     nS = world.num_states
7     acts = world.action_space

```

```

8     pi = np.zeros((nS, len(acts)), dtype=float)
9
10    # Weight mapping for stochastic policy
11    _STOCH_WEIGHTS = {
12        (1, 0): 0.7,      # right
13        (0, 1): 0.1,      # down
14        (0, -1): 0.1,     # up
15        (-1, 0): 0.05,    # left
16        (0, 0): 0.05      # stay
17    }
18
19    base = np.array([_STOCH_WEIGHTS.get(a, 0.0) for a in acts], dtype=
float)
20    base /= base.sum() if base.sum() > 0 else 1.0
21
22    stay_idx = acts.index((0, 0))
23    goal = tuple(world.target_state)
24
25    for s_idx in range(nS):
26        s_xy = world.idx_to_state[s_idx]
27        if s_xy == goal or s_xy in world.forbidden_states:
28            pi[s_idx, stay_idx] = 1.0
29        else:
30            pi[s_idx, :] = base
31
32    return pi

```

Listing 5: Stochastic policy creation with weighted actions.